

Good evening, Mohan. It's a pleasure to meet you.

My name is Ashutosh Mohanty, and I have around 5 years of experience as a DevOps Engineer, mainly working with Microsoft Azure, Terraform, Azure DevOps, CI/CD pipelines, and Infrastructure as Code.

Recently, I took a short planned career break to support my family during a health concern. Since I am the primary support for my parents, I wanted to be with them during that time. Thankfully, everything is stable now, and I am fully ready and excited to start my next opportunity.

In my previous company, I worked on the Airtel Business Portal, which was used by enterprise customers such as banks, retail companies, and other businesses. The portal allowed customers to manage services like connectivity, IoT, cloud, and communication solutions from one place.

The main goal of the project was to provide a single, easy-to-use platform where customers could view services, raise support requests, track service usage, and manage everything online. Earlier, many of these activities were handled manually or through different systems, so our project helped make the process faster, simpler, and more reliable.

Our project had different teams working together. The frontend team had around 8 developers working on the React application. The backend team had around 10 to 12 developers building services using Java and Python. We also had a QA team responsible for testing before every release. Our DevOps team consisted of 6 engineers, and we handled cloud infrastructure, CI/CD pipelines, automation, deployments, security, and release management. I worked closely with the development, QA, and infrastructure teams to ensure smooth and timely releases.

The application followed a monolithic architecture. The frontend was built using React, and the backend was developed using Java and Python. Azure SQL Database was used as the database, and all services communicated through REST APIs. The application was hosted on Microsoft Azure.

As a DevOps Engineer, my main responsibility was to automate infrastructure and deployment processes. I used Terraform to create reusable modules for Azure resources such as Resource Groups, Virtual Networks, Subnets, Network Security Groups, Virtual Machines, Load Balancers, Storage Accounts, Azure SQL Database, and Azure Key Vault. This helped us maintain the same infrastructure across Development, QA, and Production environments.

For CI/CD, we used Azure DevOps YAML pipelines. Whenever a developer created a Pull Request, the pipeline automatically started. It validated the code, checked the infrastructure, and performed security scanning using tools like tfsec and Checkov before deploying the application to different environments.

For security, we used Azure Key Vault to store secrets securely and Azure Bastion to access virtual machines without exposing them to the internet. I also worked on monitoring and logging to improve application health, quickly identify issues, and ensure stable production environments.

Overall, I enjoyed building secure, automated, and scalable DevOps solutions that helped development teams release software faster, with better quality and fewer manual efforts.

If you allow, I can explain the Landing Zone of the project.

Our application followed a monolithic three-tier architecture, where we had a Frontend Layer, Backend Layer, and Database Layer.

The application was hosted on Microsoft Azure, and before deploying the application, our team designed a secure and scalable Azure Landing Zone by following Azure best practices.

The entry point of our application was Azure Application Gateway. All customer requests first reached the Application Gateway. We enabled SSL certificates to provide secure HTTPS communication and Web Application Firewall (WAF) to protect the application from common web attacks such as SQL Injection and Cross-Site Scripting (XSS).

Behind the Application Gateway, we had two Frontend Virtual Machines where the React application was deployed. Having two frontend VMs provided better availability and ensured the application remained accessible even if one VM became unavailable.

The frontend application communicated with the backend application through an Internal Load Balancer. The Internal Load Balancer distributed traffic between two Backend Virtual Machines, improving performance and ensuring high availability without exposing the backend servers to the internet.

The backend application handled all business logic, processed customer requests, and communicated with the database whenever data needed to be stored or retrieved.

For the database layer, we used Azure SQL Database, which was a Platform-as-a-Service (PaaS) offering. Since Microsoft managed the infrastructure, patching, backups, high availability, and disaster recovery, our team could focus on application development instead of database administration.

For secrets management, such as database connection strings, application secrets, and certificates, we used Azure Key Vault. This helped us avoid storing sensitive information in the application code or configuration files.

Security Implementation in the Landing Zone

Security was one of the most important aspects while designing our Azure Landing Zone. Since our Business Portal was accessed by enterprise customers over the internet, we implemented multiple layers of security to protect the application, infrastructure, and sensitive business data.

For secure administrative access to the Virtual Machines, we used Azure Bastion. This allowed us to connect to the VMs securely through the Azure Portal without exposing SSH or RDP ports to the public internet, which significantly improved the security of our environment.

1. Azure Application Gateway with WAF

The entry point of our application was Azure Application Gateway. We enabled SSL certificates to encrypt all communication between users and the application, ensuring that data transmitted over the internet remained secure.

We also enabled Web Application Firewall (WAF) on the Application Gateway. WAF protected the application from common web attacks such as SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and

other OWASP Top 10 vulnerabilities. It continuously inspected incoming HTTP and HTTPS requests and blocked malicious traffic before it reached the application servers.

2. Azure Key Vault

All sensitive information such as database passwords, API keys, certificates, connection strings, and application secrets was stored securely in Azure Key Vault instead of being hardcoded in the application or Terraform code.

This improved security by centralizing secret management, reducing the risk of credential exposure, and making it easier to rotate or update secrets without modifying the application code.

3. Azure Bastion

For secure administration of our Virtual Machines, we used Azure Bastion.

Instead of assigning Public IP addresses to the Virtual Machines, administrators connected through Azure Bastion using the Azure Portal. This allowed secure SSH or RDP access without exposing management ports such as 22 (SSH) and 3389 (RDP) to the public internet, significantly reducing the attack surface.

4. Network Security Groups (NSGs)

We implemented Network Security Groups (NSGs) to control network traffic between different subnets.

NSGs contained inbound and outbound security rules that allowed only the required ports and protocols while blocking all unnecessary traffic. This ensured that each subnet could communicate only with the resources that were required for the application.

5. Private Network Communication

The backend Virtual Machines communicated with the database over the Azure Virtual Network instead of using the public internet. This kept internal communication secure and reduced exposure to external threats.